When Execution Conditions Become Claim-Bearing

The L2–L3 Bridge as a Bounded-Warrant Layer for AI-Relevant Execution Claims

Wiard Vasen
Independent Researcher
IJmuiden, the Netherlands

Draft v0.5 — May 2026

Abstract

Contemporary AI systems increasingly depend on execution conditions below the model layer: numeric precision, accelerator path, runtime kernel selection, batching behaviour, driver state, power policy, and environmental constraints. Existing semiconductor test, reliability, and functional-safety standards provide powerful methods for testing devices, interconnects, stress behaviour, qualification, and safety-related engineering artifacts. They do not, however, specify when such lower-layer execution conditions may support claims about AI-output boundaries. This paper introduces the *L2–L3 bridge* as the transition where coordinated execution first becomes encoded representation, and develops the Universal Provenance Layer (UPL) as a *bounded-warrant* framework for adjudicating AI-relevant execution-condition claims. The framework synthesises four philosophical and methodological traditions: Toulmin's warrant model, Dewey's warranted assertibility, defeasible reasoning in the tradition of Pollock, Dung, and Prakken, and Pearl's causal inference. The relevant unit is not the chip, runtime, or model in isolation, but the claim-bearing relation between a stated execution condition and an AI-output boundary. UPL represents this relation using the judgment form $\Gamma \vdash K \mathbin{@} B \Leftarrow W : \sigma$, requiring every checked claim to be attached to an explicit boundary, witness chain, mapping, check, ruleout procedure, and adjudication status. We formalise the framework information-theoretically using conditional claim entropy $H(K \mid W, \Gamma, B)$, the data-processing inequality, and Fano's lower bound on adjudication error rate. We also survey candidate execution conditions — mixed precision, TF32-like tensor formats, deterministic kernels, denormal handling, autotuning, and thermal regime — as domains in which bounded warrants may be constructed. This draft reports one executed forward instance, DEMO_06: a toy binary linear classifier under fp32/fp16 execution-precision contrast that yields $\sigma = \texttt{checked}$ within its stated boundary, and notes two companion witnessed instances in adjacent directions: UPL-ICE for input-conditioned execution claims and UPL-WARP for agent-routing and payment-gate decisions. All three reuse the same judgment form and witness discipline. The instances demonstrate feasibility; they do not certify hardware, generalise to production AI systems, or replace semiconductor test standards. UPL does not test whether a chip is good; it adjudicates whether a stated execution condition can carry a bounded claim about an AI-output boundary.

---

1 Introduction

Most discussions about AI begin at the level of outputs: answers, labels, decisions, prompts, or semantic interpretation. But AI behaviour does not emerge directly from meaning. It emerges through layers of execution.

At the bottom of the stack are physical and runtime execution conditions: floating-point units, memory hierarchy, accelerator selection, driver state, framework flags, kernel choice, scheduling, batching, temperature, and power state. At the top are semantic claims: interpretations about the world. Between those two extremes lies a comparatively under-specified transition: the point where coordinated execution becomes encoded representation. This paper focuses on that transition. We call it the *L2–L3 bridge*.

At **L2**, execution exists as operations, kernels, and computation graphs. At **L3**, those operations crystallise into activations, weights, attention states, and encoded vectors. The central research question is:

> *When does a lower-layer execution condition, propagating through the L2–L3 bridge, become claim-bearing for an upper-layer AI-output boundary?*

The answer cannot be "whenever a trace exists." An execution trace alone is not a behavioural claim. Nor can the answer be "whenever an output changes." An output change alone does not identify the execution condition that carried it. The missing object is a bounded warrant: an explicit relation among claim, boundary, witness, mapping, check, ruleout, and adjudication status.

The claim of this work is therefore deliberately narrow. The L2–L3 transition may be the first place where lower-layer execution conditions become structurally capable of propagating into model behaviour. This transition defines a *bounded-warrant gap*: the zone between physical/runtime execution and semantic interpretation where claims about AI behaviour must first become traceable, checkable, and bounded.

The primary contribution of this paper is methodological. We introduce a vocabulary — the bounded-warrant framework — in which execution-condition claims can be stated without overclaim. The vocabulary is not invented in isolation; it synthesises four philosophical and methodological traditions that have, over the past century, developed the conceptual scaffolding required to discipline such claims (Section 2). In addition, this draft reports one bounded toy precision-contrast instance, DEMO_06 (Section 12), whose purpose is not to establish general chip-AI propagation but to demonstrate that the UPL six-witness discipline can be executed end-to-end under a stated boundary. The framework is informed in spirit by Tanenbaum's MINIX discipline [**?**]: small, clean, safe, understandable infrastructure beneath every claim.

Outline. Section 2 situates UPL within four philosophical traditions: Toulmin's warrant model, Dewey's warranted assertibility, defeasible reasoning, and causal inference. Section 3 fixes the scope: UPL is a warrant layer, not a replacement for semiconductor test, reliability, or safety standards. Section 4 characterises the layered AI execution stack. Section 5 introduces the bounded-warrant judgment form. Section 6 presents the UPL status vocabulary. Section 7 formalises the framework information-theoretically, including Fano's lower bound on adjudication error. Section 8 concentrates on the L2–L3 bridge and presents the main figure. Section 9 surveys candidate execution conditions and calibrations. Section 10 states two postulates:

reverse inference from observed anomalies to lower-layer candidates, and toy-AI feedback as a proposed empirical entry point for the reverse direction. Section 11 sketches a path from individual demonstrations to UPL test profiles, including a brief account of the payment-decision function in companion work. Section 12 discusses open questions and reports the executed forward instance DEMO_06 along with two companion witnessed instances.

2 Theoretical Foundations

The bounded-warrant framework introduced in this paper rests on four philosophical and methodological traditions that have, over the past century, developed precisely the conceptual vocabulary needed to discipline claims about evidence, justification, and inference. Each tradition contributes one essential element to UPL's judgment form. UPL itself does not claim originality in any of these traditions; its contribution lies in their synthesis applied to a specific contemporary problem.

2.1 Toulmin's warrant model

The term *warrant* in this paper derives from Stephen Toulmin's analysis of practical argumentation [6]. Toulmin showed that every defensible argument has a structure beyond the simple syllogism: it includes a *claim* (the conclusion), *grounds* (the supporting data), a *warrant* (the inferential bridge between grounds and claim), *backing* (justification for the warrant), *qualifiers* (degree of certainty), and *rebuttals* (conditions under which the argument does not hold).

UPL's judgment form $\Gamma \vdash K@B \Leftarrow W : \sigma$ is, structurally, a formalisation of Toulmin's analysis applied to AI-infrastructure claims. The claim K corresponds to Toulmin's claim; the witness set W corresponds to the grounds; the implication $\Leftarrow$ encodes the warrant; the context Γ and boundary B together constitute the backing and rebuttal conditions; and the status σ functions as Toulmin's qualifier.

The key insight Toulmin contributed was that *every* non-trivial argument has these elements, even when they are not made explicit. A bounded-warrant framework forces them to be explicit. UPL refuses to leave the warrant implicit.

2.2 Dewey's warranted assertibility

John Dewey's *Logic: The Theory of Inquiry* [1] rejected "truth" as a useful epistemic concept for the analysis of practical inquiry. Dewey argued that truth as a property attaches to assertions only after the fact, and that operational practice requires a different concept: *warranted assertibility*. A statement is warrantedly assertible when inquiry has produced sufficient evidence to back its assertion within the conditions of that inquiry.

This is precisely UPL's interpretation of $\sigma =$ `checked`. A checked claim is not declared true; it is declared warrantedly assertible within boundary B, with witnesses W, under adjudication profile Γ. Dewey's framing protects against the characteristic failure mode of safety claims about AI systems: the slide from "no problem was found" to "there is no problem".

The distinction is operationally significant. A claim adjudicated as `checked` under one boundary may be `out_of_scope` under another. This is not a weakness of the framework; it is a feature inherited from Dewey's analysis of inquiry as a bounded, situated activity rather than a production of universal truths.

2.3 *Defeasible reasoning*

The distinction between `failed` and $\neg K$, and between `contradicted` and `failed`, descends from the literature on defeasible reasoning. John Pollock's *Knowledge and Justification* [3] established that warranted belief is intrinsically defeasible: it can be undermined by additional evidence without thereby being shown false. Phan Minh Dung's foundational work on argumentation frameworks [2] provided a formal semantics for the acceptability of arguments under attack relations, distinguishing unsupported arguments from rebutted arguments from defeated arguments.

Henry Prakken and others have applied defeasible reasoning to legal argumentation [4], where the distinction between *not proven* and *not guilty* mirrors the UPL distinction between `failed` and `contradicted`. A jury that fails to convict does not thereby establish innocence; the prosecution simply failed to discharge its burden within the trial's evidentiary boundary.

UPL inherits this discipline directly. A claim adjudicated as `failed` within boundary B has not been disproved. It has been found unsupported by the witness chain W within B. A separate inquiry, with a separate witness chain, would be required to establish $\neg K$. Negation is itself a claim, and requires its own warrant.

2.4 *Causal inference*

Reverse inference — the attempt to identify which lower-layer execution condition caused an observed AI-output anomaly — is a problem of causal inference under non-identifiability. Judea Pearl's framework [**?**] formalises the conditions under which causal claims can be defended from observational or interventional data. Donald Rubin's potential outcomes framework [5] provides a complementary perspective on causal identification under controlled comparison.

The reverse-inference postulate (Postulate 1) stated later in this paper is in spirit a Pearl-style causal claim, modulated by the bounded-warrant discipline. The Bayesian posterior in (7) is the structural form of a causal inference under candidate enumeration; the witness W^{*}_{ruleout} plays the role of excluding confounding alternative explanations. The framework does not extend Pearl's calculus, but it inherits his analytical caution about non-identifiability: many lower-layer events may produce the same observable anomaly, and the ruleout obligation must be correspondingly strong.

2.5 *UPL as a modern synthesis*

UPL does not claim originality in any of the traditions surveyed above. Its contribution lies in their synthesis applied to a specific contemporary problem: claims about AI-infrastructure conditions and their relation to AI-output boundaries. Toulmin contributes the warrant structure; Dewey contributes the rejection of universal truth in favour of bounded assertibility; Pollock, Dung, and Prakken contribute the discipline of defeasible reasoning and its formal semantics; Pearl and Rubin contribute the analytical framework for causal claims.

The synthesis is not arbitrary. Each element fills a specific need that the others do not address. Without Toulmin, the structure of warrant-bearing arguments remains implicit and unauditable. Without Dewey, statuses degenerate into truth-claims that overrun their boundaries. Without defeasible reasoning, the critical distinction between failed and disproven collapses. Without causal inference, reverse-direction claims lose disciplinary grounding.

Table 1. UPL compared with adjacent engineering questions.

Domain	Typical question	UPL question
Semiconductor test	Is the device electrically testable or defective?	Can this device/runtime condition carry a bounded AI-output claim?
Reliability qualification	Has the component passed stress or lifetime requirements?	Does a stress/environment condition become claim-bearing for boundary B?
Functional safety	Is the safety lifecycle and safety evidence adequate?	Can safety/reliability evidence be mapped to an AI-execution claim?
AI evaluation	Did the model produce output O or score S?	Is the lower-layer execution condition admissible as warrant for that output boundary?
Provenance/audit	Which artifacts and versions were used?	Is the witness chain sufficient to adjudicate claim K under boundary B?

The remaining sections of this paper formalise the synthesis and report its first executed instance, alongside brief notes on two companion witnessed instances in adjacent directions.

3 Scope: UPL Is a Warrant Layer, Not a Chip-Test Replacement

UPL should not be interpreted as a replacement for established semiconductor test, reliability, or functional-safety standards. Standards and engineering flows such as IEEE 1149.1/JTAG, IEEE 1838, JEDEC stress methods, AEC-Q100, ISO 26262, and related functional-safety information-exchange practices address test access, device qualification, reliability stress, safety lifecycle, and engineering evidence across abstraction levels. UPL addresses a different problem:

> *When a lower-layer execution condition is invoked as support for a claim about an AI-output boundary, what must be bounded, witnessed, mapped, checked, ruled out, and adjudicated before that claim is allowed?*

The relevant unit in UPL is therefore not the chip, package, board, runtime, or model considered in isolation. The relevant unit is the claim-bearing relation between an execution condition and an AI-output boundary. A chip, runtime, kernel, precision mode, thermal state, or driver flag enters UPL only insofar as it is used to support such a relation.

This scope distinction is central. UPL does not test whether a chip is good. UPL tests whether a stated execution condition can carry a bounded claim about an AI-output boundary. A successful UPL judgment therefore never means "the hardware is certified" or "the AI system is safe." It means only: under this boundary, with these witnesses, according to this profile and context, the claim has the returned adjudication status σ.

4 The Layered AI Execution Stack

We adopt a six-layer model of AI execution. Each layer corresponds to a distinct *meaning-giving act* performed on the signal as it ascends:

- **L0 Occurrence**. FPU operations, cache lines, DRAM accesses, branch predictor state, sensor/counter readings. Physical execution conditions; pre-meaning.
- **L1 Coordination**. Scheduling, memory allocation, drivers, OS, runtime resource mediation. The event becomes a resource-mediated process.
- **L2 Operation**. Kernels, ops, computation graphs, backend-selected implementations. The coordinated process becomes a named operation.
- **L3 Representation**. Activations, weights, attention, encoded state. The operation crystallises into encoded state.
- **L4 Decision**. Logits, tokens, labels, output commitment. The representation becomes a committed position.
- **L5 Interpretation**. Semantic claims, prompts, tasks, world meaning. The output becomes a claim about the world.

The meaning-making chain reads, from physical occurrence to semantic claim:

$$\text{Occurrence} \rightarrow \text{Coordination} \rightarrow \text{Operation} \rightarrow \text{Representation} \rightarrow \text{Decision} \rightarrow \text{Interpretation}.$$

The naming scheme is deliberately functional: each layer is named for what *happens* there, not for the technology used to implement it. PyTorch, TensorFlow, JAX, ONNX Runtime, accelerator firmware, and compiler backends may all instantiate *Operation* at **L2**. This decoupling matters for bounded warrants: a witness at **L2** need not commit to a specific framework to remain meaningful across re-implementations, provided the boundary B states the allowed abstraction.

The reader should note where existing research traditions sit on this stack. AI safety scholarship typically operates at **L4**–**L5**: outputs, alignment, interpretation [**?**]. Chip security research typically operates at **L0**–**L1**: timing channels, speculative execution, microarchitectural state [**? ?**]. The middle of the stack — **L2** and **L3** — is comparatively under-audited. This is the bounded-warrant gap.

5 The Bounded-Warrant Framework

The Universal Provenance Layer (UPL) introduces a judgment form that adjudicates claims about behaviour without asserting truth in general.

Definition 1 (UPL judgment)**.** A UPL judgment is an expression

$$\Gamma \vdash K \mathbin{@} B \Leftarrow W : \sigma$$

where Γ is a set of stated assumptions, rules, profiles, and allowed inferential moves; K is the claim; B is the boundary within which K is asserted; W is the witness set offered in support; and $\sigma \in \Sigma$ is the status returned by adjudication.

The judgment is not a proof and not a certification. UPL does not validate universal truth; it adjudicates bounded admissibility, in the sense developed by Dewey (Section 2.2). The framework distinguishes a candidate *UPLJudgment* (the input) from the adjudicated *UPLDecision* (the output). The same claim K may receive different statuses under different boundaries, witnesses, profiles, or assumptions.

For claims about how a lower-layer condition affects an upper-layer output boundary, we require a structured witness composed of six components:

Definition 2 (Layer-effect claim)**.** A layer-effect claim is a UPL judgment

$$\Gamma \vdash K_{\text{layer-effect}} \mathbin{@} B \Leftarrow \left(W_{\text{input}} \oplus W_{\text{exec}} \oplus W_{\text{map}} \oplus W_{\text{contract}} \oplus W_{\text{check}} \oplus W_{\text{ruleout}}\right) : \sigma$$

where the witness components are:

- W_{input} — the input, workload, seed, dataset slice, or traffic condition under which the claim is asserted;
- W_{exec} — observation of the lower-layer execution condition itself, such as precision mode, backend, kernel, driver, counter trace, temperature trace, or runtime flag;
- W_{map} — the mapping from execution condition to an upper-layer model variable or representational quantity;
- W_{contract} — the semantic or operational contract that fixes what counts as the upper-layer output boundary;
- W_{check} — the controlled comparison demonstrating whether the output boundary shifts under the stated condition;
- W_{ruleout} — the evidence excluding alternative attributions of the observed shift within B.

All six components are mandatory for status `checked`. A layer-effect claim with fewer than six required components cannot be checked; it receives `incomplete`, `unsupported`, or `out_of_scope`, depending on the failure mode. This is the discipline that prevents an execution trace alone — which contains no model-boundary information — from being claimed as supporting a behavioural conclusion.

Remark 1. Observation is not claim. Trace is not truth. No claim travels farther than its evidence.

6 The UPL Status Vocabulary

UPL adjudicates each judgment into a status. Table 2 states the vocabulary, its witness condition, and what may and may not follow. The central convention is that `checked` means "supported under this boundary, with these witnesses, according to this profile." It never means true in general — the warranted-assertibility interpretation of Section 2.2.

The crucial distinction is between `failed`, `contradicted`, and $\neg K$, inherited from the defeasible-reasoning tradition (Section 2.3). A failed check means the witness/check relation did not support K inside B; it does not mean K has been disproved. A contradicted judgment requires a positive witness chain for an incompatible claim under the same boundary. Negation is itself a claim and requires its own witness chain.

7 An Information-Theoretic Formulation

The qualitative status vocabulary of Section 6 admits an information-theoretic interpretation along lines familiar from Shannon [**?**] and the subsequent literature on mutual information [**?**].

Table 2. UPL adjudication statuses ($\sigma \in \Sigma$).

Status	Witness condition	Adjudication result
`unsupported`	No admissible witness is offered, or the offered witness is unrelated to K in B.	Claim is not admissible.
`bounded`	K, B, and the required witness obligations are well formed, but the witness chain is not yet assembled.	Claim is admissible as a test target only. No result follows.
`incomplete`	One or more required witness components are missing for the selected profile.	Claim cannot yet be adjudicated.
`pending_check`	Witness chain and check are specified, but the controlled comparison has not yet been executed.	Awaiting check.
`checked`	Required witnesses are present; check succeeds; ruleout passes within B.	Claim admitted within B only.
`failed`	Required witnesses are present; check does not support K.	K not supported by W in B. Not equivalent to $\neg K$.
`contradicted`	Witnesses support a mutually incompatible claim, or directly support $\neg K$ under the same B.	K is contradicted within B.
`inconclusive`	Check is executed but signal, mapping, or ruleout is insufficient to decide.	No adjudicated support or contradiction.
`out_of_scope`	Claim, witness, or inference lies outside B or outside the selected profile.	Claim falls outside stated scope.

7.1 Claim entropy and witness information gain

Let K be a binary random variable whose value is the truth of the claim under Γ and B. Before any witness is offered, the uncertainty about K is the prior claim entropy

$$H(K \mid \Gamma, B) \;=\; -\sum_{k \in \{0,1\}} \Pr(K = k \mid \Gamma, B)\, \log_2 \Pr(K = k \mid \Gamma, B).$$

A witness W reduces this uncertainty to the conditional entropy $H(K \mid W, \Gamma, B)$. The information gained by adjudicating W is the mutual information

$$I(K; W \mid \Gamma, B) \;=\; H(K \mid \Gamma, B) - H(K \mid W, \Gamma, B). \tag{1}$$

We say a witness is *claim-bearing* for K in B when $I(K; W \mid \Gamma, B) > \tau$ for some pre-stated threshold $\tau > 0$. The threshold is part of the boundary B, not a free parameter: B must specify what counts as adequate information transfer before any check is run.

Several UPL statuses correspond naturally:

- `checked`: $I(K; W \mid \Gamma, B) > \tau$, the update supports K, and the profile's ruleout obligations pass.
- `contradicted`: $I(K; W \mid \Gamma, B) > \tau$, but the update supports an incompatible claim under the same B.

- `failed`: the specified check does not support K, without establishing a positive incompatible claim.
- `incomplete` or `unsupported`: W is missing, malformed, or carries insufficient information for the selected claim/profile.
- `inconclusive`: $I(K; W \mid \Gamma, B)$, mapping, or ruleout is insufficient to adjudicate either support or contradiction.

The remaining distinctions concern well-formedness, boundary, and profile compliance; they are not reducible to information-theoretic quantities alone.

7.2 Inter-layer mutual information

Let X_i denote the random variable describing the state at layer **Li**. For a propagation chain $X_0 \to X_1 \to \cdots \to X_5$, the data-processing inequality [**?**] gives

$$I(X_0; X_5) \;\leq\; \min_{0 \leq i < 5} I(X_i; X_{i+1}). \tag{2}$$

The bound says: the amount of **L0** information reaching **L5** cannot exceed the narrowest channel along the path. If any single inter-layer channel has low mutual information, no fine-grained **L0** condition can survive to **L5** as a strong claim-bearing signal.

7.3 The L2–L3 transition as channel

Within this framing, the L2–L3 transition is itself a channel, denoted $\mathcal{C}_{2\to3} : X_2 \to X_3$, whose capacity $C(\mathcal{C}_{2\to3})$ bounds how much **L2**-level operational detail can be encoded into **L3**-level representational state.

Proposition 1 (Bridge bottleneck)**.** For a propagation chain $X_0 \to \cdots \to X_4$ in which $\mathcal{C}_{2\to3}$ is the unique non-trivial bottleneck,

$$I(X_0; X_4) \;\leq\; C(\mathcal{C}_{2\to3}).$$

Proposition 1 is a corollary of (2) and provides a precise sense in which the L2–L3 bridge can be informationally decisive. Where the bridge is narrow — e.g. aggressive quantisation, dimensionality collapse, attention masking — lower-layer execution conditions cannot propagate finely. Where it is wide — e.g. high-precision matmul with preserved fan-out — they may. The framework does not assert either condition holds in any specific deployment; it states the structural relation.

7.4 Lower bound on error rate from Fano's inequality

Information-theoretic constraints also give us a lower bound on *how often* an adjudication can be in error. Fano's inequality [**? ?**] relates the conditional entropy $H(K \mid W)$ to the probability P_e of incorrect adjudication. For a binary claim $K \in \{0, 1\}$:

$$H(K \mid W, \Gamma, B) \;\leq\; H_b(P_e), \tag{3}$$

where $H_b(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ is the binary entropy function. Equivalently,

$$P_e \;\geq\; H_b^{-1}\big(H(K \mid W, \Gamma, B)\big), \tag{4}$$

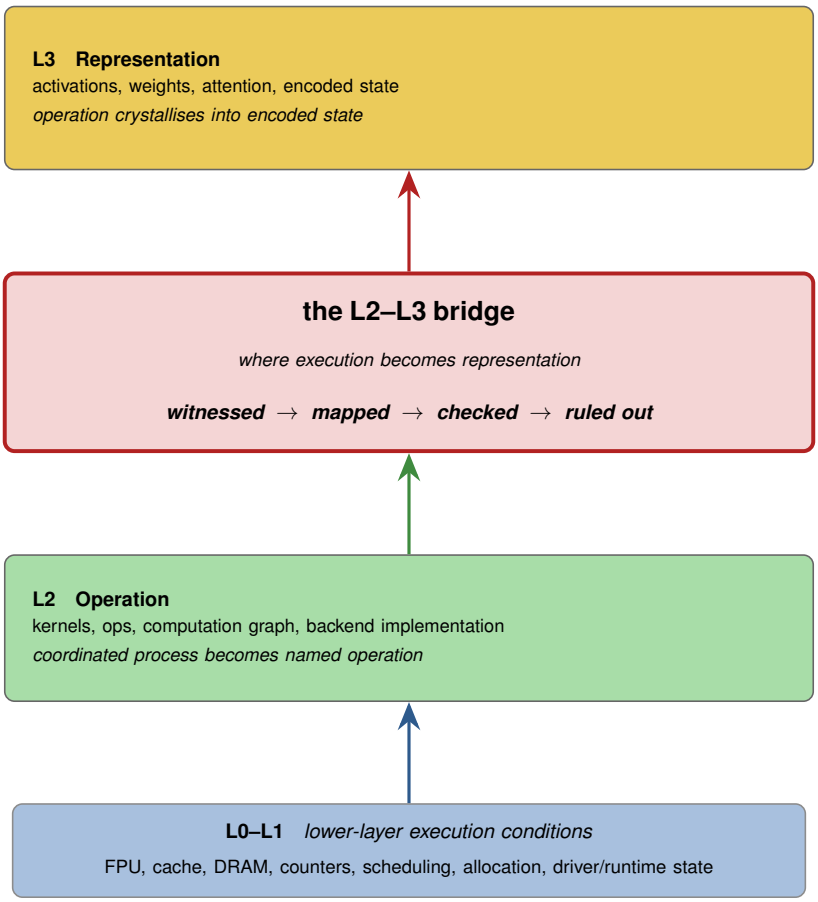

Figure 1. The L2–L3 bridge as a bounded-warrant zone. Lower-layer execution conditions (**L0–L1**) traverse **L2** *Operation* before crystallising at **L3** *Representation*. The bridge is the locus at which an execution condition can become structurally capable of carrying a bounded claim about an AI-output boundary. The four operations *witnessed* → *mapped* → *checked* → *ruled out* summarise the bounded-warrant discipline that any layer-effect claim must satisfy across this zone.

where H_b^{-1} denotes the inverse of H_b on $[0, 1/2]$.

The interpretation is operational: if the witness leaves the claim sufficiently uncertain — that is, if $H(K \mid W, \Gamma, B)$ is large — then no adjudication procedure can drive P_e below the bound. Adjudicating a claim with a weak witness will be wrong some of the time, and the lower bound on that error rate is set by information, not by adjudicator ingenuity. This is one of the strongest formal motivations for the bounded-warrant discipline: constructing W with high mutual information is not optional, because P_e is rate-limited by $H(K \mid W)$ regardless of how the adjudication is implemented.

Remark 2. Fano's bound applies to the adjudicator viewed as a decoder of K from W. UPL is not a decoder in the cryptographic sense, but the structural relation holds: insufficient information cannot be compensated for by procedure.

8 The L2–L3 Bridge

Figure 1 situates the bridge within the stack and labels the bounded-warrant framework as a four-stage discipline.

Three properties make the L2–L3 transition the natural locus for bounded warrants:

1. **First point of representational identity.** Below **L3**, no entity in the computation is identifiable as "a weight" or "an activation" except through the framework's operational interpretation. At **L3**, tensors acquire positional and functional role within the model.
2. **Bottleneck candidacy.** Per Proposition 1, the **L2**-**L3** channel is a candidate bottleneck for information propagation. Whether it is the binding constraint depends on the specific deployment, but it is the layer at which precision, quantisation, kernel choice, and architectural choices most directly intersect.
3. **Witness constructibility.** Witnesses at **L0** are abundant (hardware counters, performance traces, microarchitectural probes) but often disconnected from model boundaries. Witnesses at **L5** are abundant (output logs, evaluations, prompts) but disconnected from physical and runtime execution. The L2–L3 bridge is the layer at which a witness can plausibly connect both ends — where W_{exec} and W_{map} can be defined together.

Figure 2 extends the bridge model with explicit operator structure and bidirectional reversibility. The L2.5 zone is not a new physical layer; it is a bounded reasoning zone within which forward and reverse claims can be named, measured, and adjudicated under the same witness discipline.

9 AI-Relevant Execution Conditions and Calibrations

The bounded-warrant framework is motivated by, but does not assume, real instances of lower-layer execution conditions affecting model behaviour. Several configurable *calibrations* are documented as affecting numerical traces, reproducibility, or boundary-near outputs. We list them as candidate domains for which UPL profiles should ultimately be tested. The list is not a catalogue of proven AI-semantic causes.

Mixed-precision arithmetic (FP32 $\leftrightarrow$ FP16/BF16). Mixed-precision training and inference reduce numerical precision of intermediate values, typically with master-weight preservation [**?**]. The lower-layer effect is a change in floating-point rounding behaviour; the L2 effect is often a change in kernel selection; the L3 effect is a perturbation of activations whose magnitude depends on operator dynamic range. The witness chain $W_{\text{input}} \oplus W_{\text{exec}} \oplus W_{\text{map}} \oplus W_{\text{contract}} \oplus W_{\text{check}} \oplus W_{\text{ruleout}}$ is in principle constructible for a specific input set on which two precision regimes diverge at a stated output boundary.

Tensor Float-32 and related tensor formats. TF32 [**?**] retains FP32 dynamic range while using a shorter mantissa for tensor operations, accelerating matmul at the cost of reduced precision in the inner product. Runtime libraries and frameworks may expose such behaviour through configuration flags or backend policy. In UPL terms, the flag or backend policy is not itself the claim; it is a candidate execution condition that may or may not become claim-bearing for a given output boundary.

Deterministic versus non-deterministic kernels. Several deep-learning kernels admit multiple implementations that return numerically distinct outputs on identical inputs due to non-associativity of floating-point addition under varying reduction orders [**?**]. Runtime libraries may expose deterministic-mode settings that constrain kernel choice. The lower-layer effect is a change in reduction order or implementation; the L3 effect is a perturbation in

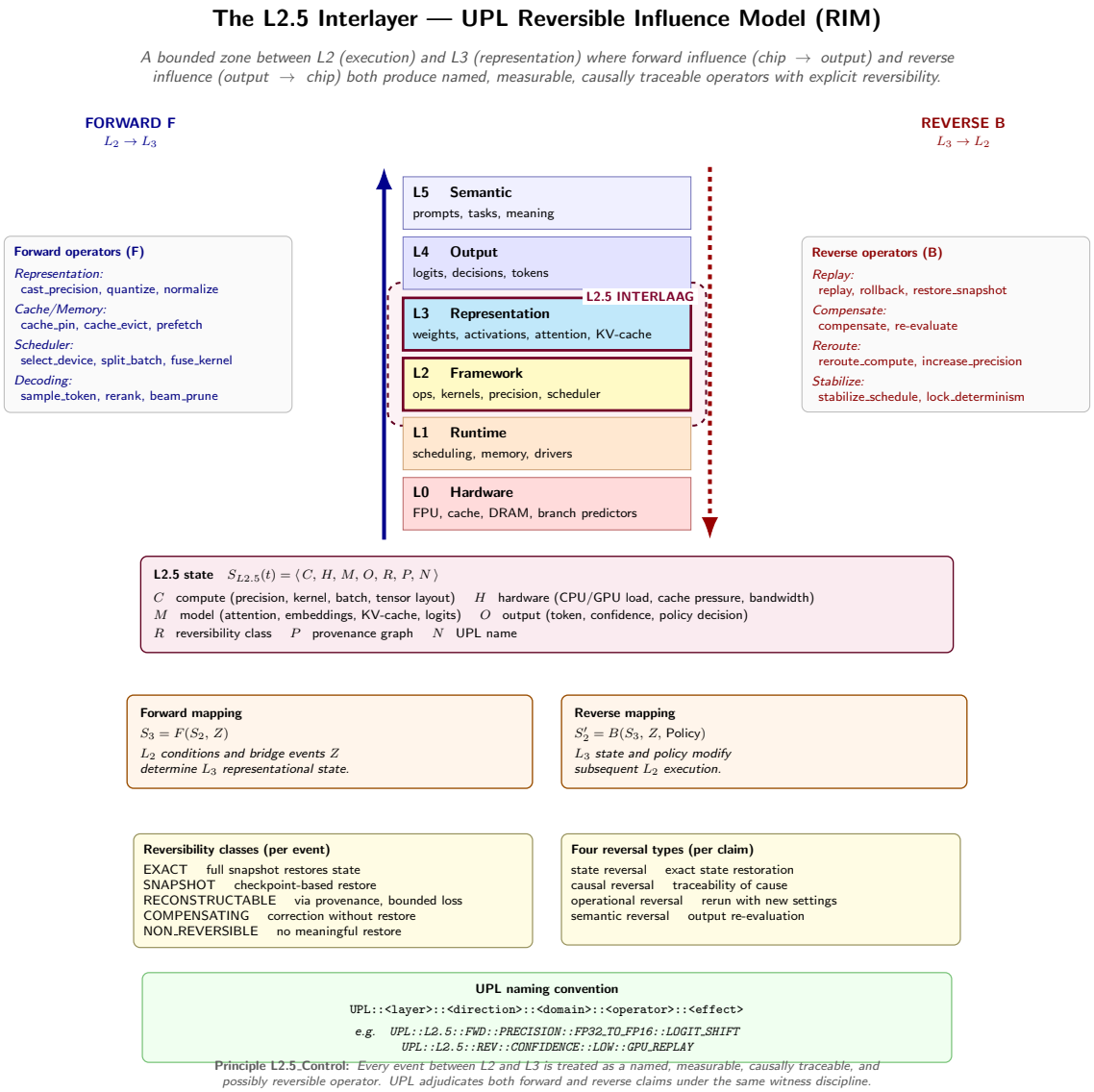

Figure 2. The **L2**.5 Interlayer — UPL Reversible Influence Model (RIM). The bounded zone between **L2** (execution) and **L3** (representation) is where forward influence (chip/runtime → output) and reverse influence (output → chip/runtime) are represented as candidate named, measurable, causally traceable operators with explicit reversibility classes. The L2.5 state $S_{L2.5}(t) = \langle C, H, M, O, R, P, N \rangle$ captures compute, hardware/runtime, model, output, reversibility class, provenance, and UPL name. Forward and reverse mappings share the same six-witness discipline. Every event between **L2** and **L3** is treated as a named, measurable, causally traceable, and possibly reversible operator (*Principle L2.5_Control*).

computed activations; the L4 effect, in boundary-near cases, may be a flipped classification or token decision.

Denormal flush-to-zero. Many FPUs offer a *flush-to-zero* mode in which subnormal floating-point values are replaced by zero. The lower-layer effect is a change in rounding behaviour for very small values. Propagation through **L2**–**L3** depends on whether training or inference produces denormals in the relevant operator, which itself depends on architecture, initialisation, scale, and activation distribution.

Memory layout and kernel autotuning. Frameworks and libraries can select kernel implementations based on input shape, memory layout, device state, and benchmarking policy. The selected implementation may differ across configurations or runs, leading to small numerical differences. The relevant condition sits at **L1–L2**: the policy that selects the operation implementation. A UPL claim would require not merely observing the policy, but mapping it to the representational and output boundary under test.

Thermal regime and operating temperature. A chip's operating temperature T is a continuous calibration with well-documented influence on lower-layer error rates and margins. Two physical mechanisms apply. First, Johnson–Nyquist noise gives a thermal floor on the voltage fluctuation across any resistive element of resistance R in bandwidth B:

$$\langle v_n^2 \rangle \;=\; 4k_B T R B, \tag{5}$$

where k_B is the Boltzmann constant. Higher T raises the noise floor, reducing margin against signal-discrimination errors. Second, thermally activated failure rates — charge-leakage, single-event upsets, and certain refresh-related DRAM errors — follow an Arrhenius dependence:

$$\lambda(T) \;=\; \lambda_0 \exp\!\left(-\frac{E_a}{k_B T}\right), \tag{6}$$

where λ_0 is a baseline rate and E_a is the activation energy of the relevant failure mode. Large-scale field studies of production DRAM [**?**] confirm that error rates rise strongly with operating temperature, and architectural studies have documented disturbance-error mechanisms whose rate depends on both temperature and access pattern [**?**]. Within the present framework, T is a lower-layer execution condition whose configuration is partially within operator control (cooling, workload placement, throttling policy) and partially outside it (ambient temperature, transient bursts). A layer-effect claim that implicates a thermal regime would require W_{exec} to include a temperature trace and W_{ruleout} to exclude attribution to concurrent calibrations.

Each calibration above is, in UPL terms, a hypothesis about a candidate signal. The survey itself does not claim any listed calibration to be a witnessed, mapped, checked, and ruled-out cause of any specific AI behaviour under any specific boundary. The one executed instance reported in this draft is the explicitly bounded DEMO_06 precision-contrast case in Section 12. The bounded foundational claim of UPL [**? ?**] is precisely that: for specific behaviours, under stated boundaries, with explicit mapping and check, a lower-layer execution trace *can* be made claim-bearing for an upper-layer output. The reduction is local, bounded, witnessed.

10 Postulates of Reverse Inference and Toy-AI Feedback

The framework presented so far runs in the forward direction: from a stated execution condition W_{exec} through the bridge to an upper-layer output boundary W_{check}. Two symmetric questions arise. The first concerns *reverse inference* — tracing from an observed AI anomaly back to a candidate lower-layer execution condition. The second concerns *toy-AI feedback* — the observation that a small, controlled AI computation itself produces a measurable signature at the chip/runtime level, which provides an empirical entry point to the framework.

10.1 Reverse inference

> *Given an observed anomaly at the AI-output boundary, can the same witness vocabulary be used in reverse, to identify a candidate lower-layer execution condition that may have produced it?*

We do not assume this is possible in general. We postulate it as a research direction, subject to the same bounded-warrant discipline as the forward direction, and grounded in the causal-inference tradition of Section 2.4.

Postulate 1 (Reverse inference)**.** Let K^* be a claim of the form "output anomaly A at **L4** was carried by lower-layer execution condition $c \in \mathcal{C}$" where $\mathcal{C}$ is a finite catalogue of candidate conditions. Then under appropriate assumptions Γ, a reverse-direction UPL judgment

$$\Gamma \vdash K^* \,@\, B \;\Leftarrow\; \left(W_{\text{anomaly}} \oplus W^*_{\text{exec}} \oplus W^*_{\text{map}} \oplus W_{\text{contract}} \oplus W^*_{\text{check}} \oplus W^*_{\text{ruleout}}\right) \;:\; \sigma$$

may be adjudicated with the same status vocabulary Σ as the forward direction, provided:

- the anomaly is observable at **L4** or higher and admits a stated boundary B;
- the candidate catalogue $\mathcal{C}$ is finite, enumerated, and sufficiently disjoint for the selected profile;
- the inverse map W^*_{map} can be constructed from **L3** representational state back to a specific candidate $c \in \mathcal{C}$;
- W^*_{ruleout} excludes the remaining candidates in $\mathcal{C} \setminus \{c\}$ under B.

A useful concrete form for W^*_{map} is Bayesian. If $\mathcal{C} = \{c_1, \ldots, c_n\}$ with prior $\Pr(c_i)$, the posterior over candidates given the observed anomaly A is

$$\Pr(c_i \mid A, \Gamma, B) \;=\; \frac{\Pr(A \mid c_i, \Gamma, B)\ \Pr(c_i)}{\sum_{j=1}^{n} \Pr(A \mid c_j, \Gamma, B)\ \Pr(c_j)}. \tag{7}$$

A reverse-direction judgment reaches `checked` for candidate c_i when the posterior concentrates on c_i above a stated threshold, and when W^*_{ruleout} supplies witnesses that the remaining candidates are excluded under B. Equation (7) is not a procedure but a constraint on what counts as a defensible reverse-inference judgment.

The postulate is consequential. It says the bounded-warrant framework is in principle bi-directional, even though the information-theoretic challenges are asymmetric. Forward inference faces the data-processing inequality (2): information may be lost in propagation, so a strong lower-layer signal may not survive to be witnessed at **L4**. Reverse inference faces non-identifiability [**?**]: many lower-layer events may produce the same observable anomaly, so W^*_{ruleout} must work harder.

Reverse inference connects naturally to causal-inference frameworks [5**?**] and to the literature on information-flow tracking in security [**?**]. We do not extend the formal connection in this paper. We note only that the bounded-warrant vocabulary is symmetric in form, and asymmetric only in the difficulty of constructing the required witnesses.

10.2 Toy-AI feedback as an empirical entry point

Reverse inference (Postulate 1) carries a constructive obstacle: how is W^*_{map} to be defined in practice? The forward direction draws on hardware counters, runtime metadata, and

framework instrumentation; the reverse direction needs a procedure that takes an observed output anomaly and produces a posterior over lower-layer candidates. We postulate that the simplest tractable setting in which this map is constructible is the case of a deterministic toy AI.

Postulate 2 (Toy-AI feedback)**.** Let $\mathcal{M}$ be a deterministic, fully specified computation — a *toy AI* — confined within hardware/runtime boundary B. Let $X_3(\mathcal{M}, t)$ denote $\mathcal{M}$'s **L3** representational state at time t, and let $S(\mathcal{M}, t) \in \mathcal{S}$ denote the lower-layer signature produced as a by-product of $\mathcal{M}$'s execution — comprising thermal trajectory, instantaneous power draw, performance-counter readings, cache footprint, and memory-bus timing. Then under appropriate Γ and over a stated horizon,

$$I\big(S(\mathcal{M}, t)\,;\, X_3(\mathcal{M}, t) \,\big|\, \Gamma, B\big) \;>\; 0. \tag{8}$$

The postulate asserts that the lower-layer signature is non-trivially informative about the toy AI's internal state. This is not a strong claim — the right-hand side need not be large — but it is sufficient to make reverse inference non-vacuous in at least one setting. By Fano's inequality (3), if the conditional entropy $H(S \mid X_3, \Gamma, B)$ remains high, any reverse adjudicator retains a non-zero lower bound on error. Estimating $I(S; X_3)$ therefore quantifies how informative the signature can be, without by itself guaranteeing successful reverse adjudication; an instrumented toy-AI setting nevertheless provides a concrete construction of W^*_{exec}, W^*_{map}, and ultimately W^*_{ruleout}.

The toy-AI setting is empirically tractable in a way that production-scale systems are not. The model is small enough to be fully characterised; the boundary B is small enough to be instrumented; the computation is deterministic enough that repeated execution yields a statistical estimate of $I(S; X_3)$. The framework's reverse-inference postulate is therefore reduced, in its first instantiation, to a controlled measurement problem: can a lower-layer signature be shown to carry τ bits of information about a toy AI's representational state within stated B?

Remark 3. Postulate 2 does not assume the converse for production systems. Scaling from toy-AI feedback to bounded reverse inference on production AI is left open. The postulate serves as a proposed empirical entry point for the reverse direction, not as a universal claim. The first executed instance reported in this paper is forward rather than reverse: it tests a toy precision-contrast layer-effect claim (Section 12).

11 Toward UPL Test Profiles

A single checked instance is useful, but it is not yet a standard. For UPL to become reusable, claims must be grouped into profiles that specify allowed claims, required witnesses, forbidden interpretations, and adjudication rules. A profile turns the general judgment form into an executable test family.

Every profile should specify:

- required witness components and minimum metadata;
- allowed claim forms and forbidden claim forms;
- boundary requirements and threshold conventions;
- accepted check procedures and ruleout obligations;

Table 3. Candidate UPL profiles.

Profile	Purpose	Typical forbidden claims
UPL-P0 Reproducibility	Minimal toy reproducibility: code, environment, input, seed, output trace, judgment file.	Hardware causation, production safety, general reliability.
UPL-P1 Numeric execution	Precision, datatype, rounding, quantisation, and arithmetic-mode claims.	General model quality or universal precision behaviour.
UPL-P2 Runtime/kernel	Deterministic kernels, backend selection, batching, reduction order, scheduling-sensitive execution.	Claims about silicon defects or long-term reliability.
UPL-P3 Accelerator path	CPU/GPU/TPU/NPU/backend-path comparisons under matched workload boundaries.	Certification of accelerator correctness outside B.
UPL-P4 Environmental execution	Temperature, throttling, power cap, clocking, memory pressure, sustained-load conditions.	Broad thermal reliability or field-failure claims without external qualification.
UPL-P5 Production audit	Signed artifacts, runtime attestation, deployment provenance, exception logs, production-boundary adjudication.	Safety certification or legal compliance unless separately witnessed.

- permitted statuses and failure modes;
- reproducibility requirements, including independent rerun rules.

In this roadmap, DEMO_06 is best understood as an early UPL-P1 instance: a bounded numeric-execution claim involving fp32 versus fp16 precision contrast in a toy classifier. It is not yet a profile, but it is a profile seed. A future sequence might include TF32-like mode contrast, deterministic versus non-deterministic kernels, batching and reduction-order effects, CPU versus GPU execution path, thermal/throttling boundaries, and a first reverse inference toy case. Each instance should preserve the same judgment shape:

$$\Gamma \vdash K \mathbin{@} B \Leftarrow W : \sigma.$$

The next milestone after additional demonstrations is independent reproduction: another party should be able to run the profile on a different machine, either obtain the same boundary-status relation, or report a boundary change explicitly. Only then does UPL begin to move from paper vocabulary to shared test practice.

Beyond execution conditions: the payment-decision function. While the profiles in Table 3 focus on execution-condition claims, the bounded-warrant framework extends naturally to

other classes of AI-infrastructure claims that require adjudication before action. As an illustrative example, an agent-routing-and-payment application (UPL-WARP) has been developed in companion work [8], adjudicating claims of the form

$$\Gamma_{\text{WARP}} \vdash K_{\text{handoff/payment}} \,@\, B \Leftarrow W : \sigma.$$

Here $K_{\text{handoff/payment}}$ is the claim that an AI agent may delegate bounded task S to another agent and authorise mock payment P; B encodes mandate, budget, allowed agents, allowed task types, and payment mode; and W comprises mandate, route, quote, contract, check, and ruleout witnesses. A deterministic policy engine maps the adjudication status σ to a payment action via a tabulated function $\pi : \Sigma \to \text{Actions}$, with typical entries

$$\begin{aligned}
\pi(\texttt{checked}) &= \texttt{release_allowed},\\
\pi(\texttt{bounded}) &= \texttt{human_approval_required},\\
\pi(\texttt{pending_check}) &= \texttt{pending},\\
\pi(\texttt{inconclusive}) &= \texttt{hold},\\
\pi(\texttt{failed}) &= \texttt{denied},\\
\pi(\texttt{contradicted}) &= \texttt{denied},\\
\pi(\texttt{out_of_scope}) &= \texttt{denied},\\
\pi(\texttt{incomplete}) &= \texttt{blocked_missing_witnesses}.
\end{aligned}$$

The critical architectural property is that the language model proposes; UPL adjudicates; the deterministic policy engine authorises. The LLM never moves funds or releases payment directly. This separation of proposal from authorisation is a structural safety property inherited from the bounded-warrant discipline. A first witnessed instance (DEMO_WARP_01, $\sigma =$ `checked`, mock ledger) demonstrates that the discipline applies; like the execution-condition instances in this paper, it does not generalise beyond its stated boundary.

A second adjacent direction, UPL-ICE for input-conditioned execution claims [7], adjudicates claims of the form $\Gamma \vdash K_{I \to E} @ B \Leftarrow W : \sigma$, where I is AI input behaviour (prompt length, batch shape, agent-action pattern) and E is an execution condition (memory pressure, kernel selection, latency regime) induced by that input. The first witnessed instance in that direction (DEMO_ICE_01, $\sigma =$ `checked`) tests whether increasing input context length beyond a declared threshold induces a memory-pressure regime within a stated boundary. Both UPL-WARP and UPL-ICE reuse the judgment form, witness structure, and status vocabulary of this paper unchanged. They illustrate that the bounded-warrant discipline is not specific to hardware or runtime claims; the same formalism applies wherever bounded claims about AI-related events require adjudication.

12 Discussion

Scope and first witnessed instance. This paper presents the framework within which AI-relevant execution-condition claims can be made. The next research step described in earlier drafts — the construction of a single, fully witnessed instance of a layer-effect claim across the **L2– L3** bridge — has now been executed.

Under a local-machine, single-venv boundary using PyTorch 2.12.0 and a deterministic toy binary linear classifier ($d = 16$), same-precision determinism held in both fp32 and fp16.

The cross-precision contrast yielded zero control flips and one margin flip. Under the declared boundary B, the UPL layer-effect claim

> *"Changing only the execution precision from fp32 to fp16 in the **L2** dot-product operation changes the measured margin representation $z = \mathbf{w} \cdot \mathbf{x} + b$ enough to cross the predefined binary decision boundary for at least one input in X_{margin}, while X_{control} remains invariant"*

was adjudicated with $\sigma = \texttt{checked}$. The full witness chain $\left(W_{\text{input}} \oplus W_{\text{exec}} \oplus W_{\text{map}} \oplus W_{\text{contract}} \oplus W_{\text{check}} \oplus W_{\text{ruleout}}\right)$ is available in the project repository [**?**].

This establishes the feasibility of the UPL witness discipline in a toy precision-contrast setting. It does not establish broader claims about large language models, production AI systems, chip defects, semiconductor reliability, or hardware behaviour generally. The bounding is intentional and follows directly from the framework's discipline.

Three witnessed instances across three domains. At the time of this draft, the bounded-warrant discipline has been demonstrated in three distinct domains: the L2–L3 bridge (DEMO_06, $\sigma = \texttt{checked}$); input-conditioned execution (DEMO_ICE_01, $\sigma = \texttt{checked}$, see [7]); and agent routing with mock payment-gate decisions (DEMO_WARP_01, $\sigma = \texttt{checked}$, see [8]). Each instance was adjudicated under the same judgment form $\Gamma \vdash K@B \Leftarrow W : \sigma$, with the canonical six-witness structure, the same status vocabulary, and the same bounded-warrant principle. The three instances together suggest that UPL's contribution is not specific to hardware-execution claims but extends to a wider class of AI-infrastructure adjudication problems. The instances are preliminary and do not constitute a general theory; they are proof-of-discipline rather than proof-of-coverage.

Relation to existing fields. UPL is not an AI safety framework. It is not a chip security framework. It is not a semiconductor qualification framework. It is a provenance and adjudication vocabulary that may inform all three. AI safety has tools for talking about output behaviour; chip security has tools for talking about microarchitectural state; semiconductor test and reliability have tools for talking about device access, defects, stress, and qualification. UPL asks a different question: when does evidence from one of these domains become an admissible warrant for a bounded AI-output claim?

What this paper does not claim. The framework does not assume that lower-layer execution conditions matter for AI semantics. It does not assert that any specific calibration in Section 9 causes any specific behaviour under any specific boundary. It does not claim that reverse inference is achievable in any specific case. It does not replace JTAG, DFT, ATE, JEDEC stress testing, AEC qualification, ISO 26262 safety lifecycle analysis, or production audit controls. It claims only that a bounded vocabulary exists in which AI-relevant execution-condition claims can be stated, adjudicated, and limited.

Open questions. At least five:

- What measurement instruments can construct W_{map} across the L2–L3 bridge for current production frameworks?
- Under what conditions is W_{ruleout} constructible without exhaustive enumeration of $\mathcal{C}$?
- How does the framework compose across multiple bridges (e.g. **L0**-**L2** and **L2**-**L3** jointly)?

- Does the reverse-inference postulate (Postulate 1) admit a constructive proof under bounded assumptions, or does it require case-by-case argument?
- What is the smallest profile schema that allows independent UPL reproductions without smuggling in unbounded hardware or AI claims?

A note on terminology. We have deliberately avoided using *fault*, *attack*, *breach*, *certification*, and *compliance* as operative labels for the claims made in this framework. Where such terms appear in the paper, they refer to neighbouring literatures or to distinctions the framework deliberately does not claim. These terms carry witness commitments the present framework does not yet support: a fault asserts a failure of specification; an attack asserts intent and control; a breach asserts a witnessed crossing of a security boundary; certification and compliance assert satisfaction of an external standard. The framework presented here talks instead of *execution conditions*, *candidate signals*, *boundaries*, *witnesses*, and *layer-effect claims*. The terminological restraint is part of the bounded-warrant discipline.

Acknowledgements

This work proceeds in the spirit of small, clean, safe, understandable infrastructure that has marked Andrew Tanenbaum's contributions to operating systems pedagogy. Its conceptual vocabulary draws on a longer tradition: the warrant model of Stephen Toulmin, the warranted-assertibility framing of John Dewey, the defeasible-reasoning tradition of John Pollock, Phan Minh Dung, and Henry Prakken, and the causal-inference framework of Judea Pearl and Donald Rubin. Errors and infelicities remain the author's own.

References

[1] John Dewey. *Logic: The Theory of Inquiry*. Henry Holt and Company, New York, 1938.

[2] Phan Minh Dung. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2): 321–357, 1995.

[3] John L. Pollock. *Knowledge and Justification*. Princeton University Press, Princeton, NJ, 1974.

[4] Henry Prakken. An abstract framework for argumentation with structured arguments. *Argument and Computation*, 1(2):93–124, 2010.

[5] Donald B. Rubin. Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology*, 66(5):688–701, 1974.

[6] Stephen E. Toulmin. *The Uses of Argument*. Cambridge University Press, Cambridge, UK, 1958. Updated edition 2003.

[7] Wiard Vasen. UPL-ICE: Input-Conditioned Execution Warranting. First witnessed instance demo_ICE_01. https://github.com/wiard/upl-ice, 2026. Companion work. Tag: demo_ICE_01-checked-v1.0.

[8] Wiard Vasen. UPL-WARP: Warranted Agent Routing and Payment. First witnessed instance demo_WARP_01. https://github.com/wiard/upl-ice/tree/main/modules/warp, 2026. Module within upl-ice. Tag: demo_WARP_01-checked-v1.0.